

LAB 4- Generate Performance of worst case of Lab 1, Lab 2 and Lab 3

Theory:

Lab 4 consolidates the worst and best case performances of three elementary sorting algorithms — **Insertion Sort, Bubble Sort, and Selection Sort** — studied individually in Labs 1, 2, and 3. By plotting their comparison counts on a unified graph, we can directly evaluate their relative efficiency under identical input conditions.

Algorithm Complexity Summary

Algorithm	Best Case	Condition	Worst Case	Condition
Insertion Sort	$O(n)$	Already sorted	$O(n^2)$	Reverse sorted
Bubble Sort	$O(n)$	Already sorted	$O(n^2)$	Reverse sorted
Selection Sort	$O(n^2)$	Any order	$O(n^2)$	Any order

Worst Case Behavior

In the worst case (reverse-sorted input), both insertion and bubble sort must perform the **maximum number of comparisons and swaps** as every element is out of place. Selection sort reaches its worst case under **any input order** since it always scans the entire unsorted portion to find the minimum, making it input-order insensitive.

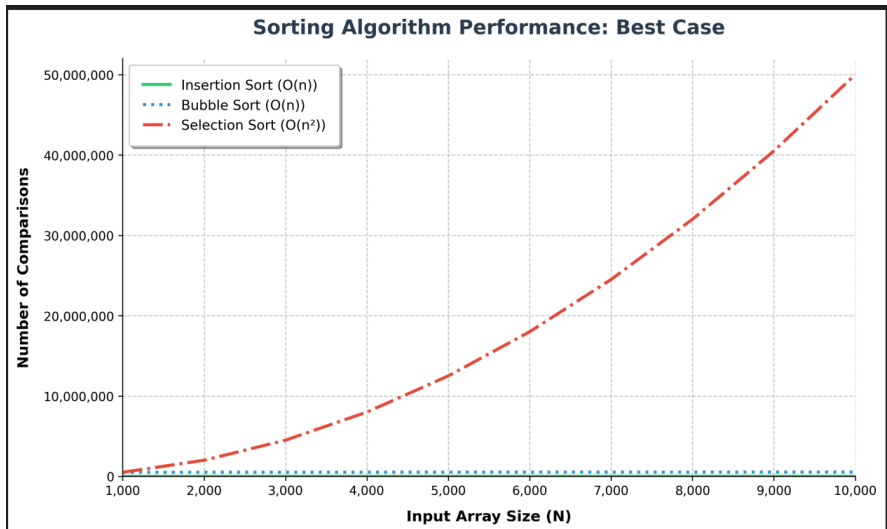
Best Case Behavior

Insertion and bubble sort exploit the **early termination** property when input is already sorted — insertion sort needs only $n-1$ comparisons with no shifts, and bubble sort exits after one pass with zero swaps. Selection sort gains **no benefit** from sorted input as it still performs $n(n-1)/2$ comparisons regardless, making it strictly $O(n^2)$ in all cases.

Graphs and Functional Growth:

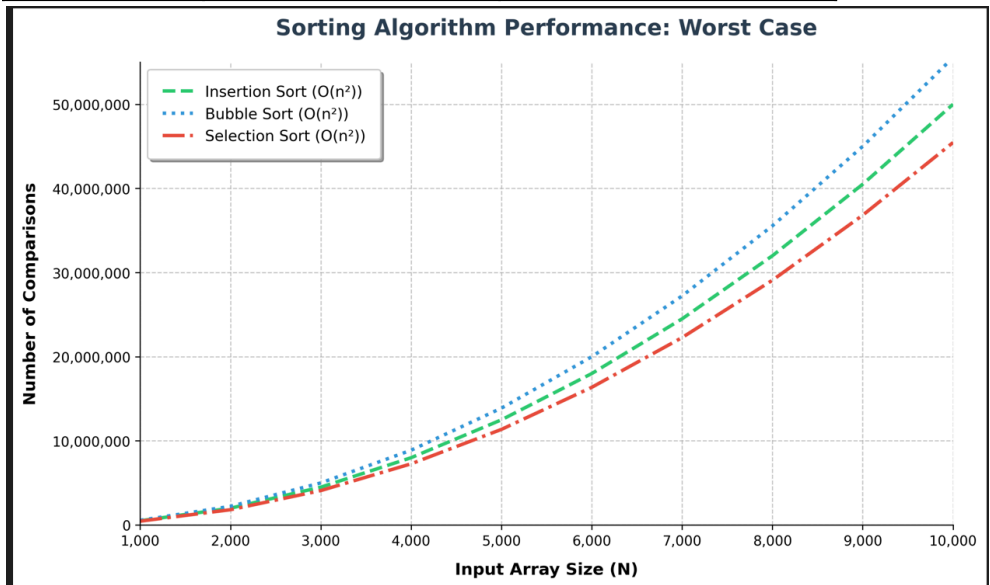
Best Case:

Algorithm	Input Condition	Time Complexity
Insertion Sort	Already Sorted (Ascending)	$O(n)$
Bubble Sort (Optimized)	Already Sorted (Ascending)	$O(n)$
Selection Sort	Any order	$O(n^2)$



Worst Case:

Algorithm	Input Condition	Time Complexity
Insertion Sort	Random Order	$O(n^2)$
Bubble Sort	Random Order	$O(n^2)$
Selection Sort	Random Order	$O(n^2)$



Conclusion:

This comparison shows that no single algorithm is perfect for every situation. In the best-case scenario, both insertion and bubble sort run in $O(n)$ time, making them way better than selection sort, which gets stuck at $O(n^2)$ even if the data is already sorted. You can see this on the graphs: insertion and bubble sort stay almost flat, while selection sort curves upward heavily.

In the worst-case scenario, all three slow down to $O(n^2)$. However, insertion sort still ends up doing fewer actual comparisons than bubble sort because its inner loop can break early, whereas selection sort's operation count stays exactly the same. Overall, insertion sort adapts the best to nearly sorted data, while selection sort doesn't adapt at all. These limitations are exactly why we don't use these elementary sorts for massive datasets and instead rely on faster ones like merge sort or quick sort.